ThoughtSpot: Developer / Embedding Surface, APIs, Integrations & MCP

Research for the Spiff UX team. Sections marked [VERIFIED] are sourced from live ThoughtSpot / Anthropic / modelcontextprotocol.io documentation with URLs. Sections marked [DESIGN REASONING] are my own opinionated synthesis for the Spiff mockup — no ThoughtSpot feature is invented in the verified sections.

Researched 2026-08-31.

---

1. ThoughtSpot Everywhere / Embedded Analytics [VERIFIED]

Sources: thoughtspot.com/product/embedded-analytics · developers.thoughtspot.com (/, /docs/tsembed,
/docs/embed-spotter, /docs/spotdev-portal)

Visual Embed SDK — five named embed classes, each a different granularity of the product:

Component	Embeds
SearchEmbed	Search page with a pre-selected data source
SearchBarEmbed	Search bar + data panel only
LiveboardEmbed	One visualization, or a full Liveboard
SpotterEmbed	The Spotter AI-analyst conversation
AppEmbed	Full ThoughtSpot app, or specific pages

All initialise via one init() taking thoughtSpotHost and authType. Frameworks: React, Angular, vanilla TS/JS, plus mobile SDKs (React Native, Flutter, Swift, Android).

SpotterEmbed takes worksheetId; Spotter 3 adds 'auto_mode', letting the agent pick its own data source — this requires administrator enablement on embedded instances, so loose scoping is an opt-in governance decision, not a default. Also hideSampleQuestions, disableSourceSelection.

Developer portal: Home (pictorial map of embeddable features) → Visual Embed SDK docs + Playground (live preview for Search, Liveboards, Visualizations, full app) → REST API v1 and v2.0 Playgrounds → Customizations (Styles, Custom actions, Security settings, Link settings). Plus a Theme Builder.

Custom actions: two shapes — an action that "triggers a callback to your host application", or one that "invoke[s] a URL to send ThoughtSpot data". Marketed as "push data to Slack, Jira, or your own backend straight from embedded components," plus writebacks and workflows.

Named agents: Spotter (AI analyst), SpotterModel (semantic modelling, human-in-the-loop validation), SpotterViz (dashboards), SpotterCode (generates "production-ready authentication, embed configuration, and integration patterns"). Platform claims: multi-tenant, row-level security, SSO/SAML, audit trails, SOC 2 / GDPR / HIPAA-ready. Positioning is explicitly deterministic — governed by the semantic layer rather than "probabilistic LLM outputs," with "every answer fully traceable and auditable."

---

2. REST API v2 [VERIFIED]

Source: developers.thoughtspot.com/docs/rest-apiv2-reference · /docs/connections

Resource groups: Users/Groups/Roles/Auth; Search data, Liveboards, Answers, Reports; Metadata,
TML import-export, Connections, DBT; object permissions, column-level security, sharing; Orgs,

Tags, Schedules, Logs, System config; email templates, style settings, custom actions, custom calendars; AI/Spotter agent conversations, Webhooks (Beta), Version control, Variables (Beta).

Token model — three types, and the interesting bit for Spiff: Full Access Token ("grants access to a full application session"); Object Access Token (restricted to specific metadata objects); Session Token (refresh from an existing session). Tokens support validation, revocation, and custom token generation with pre-defined RLS rules — so "a token narrower than the user" is already first-class here.

The v2.0 Playground offers endpoint docs, live "Try it out", and downloadable code samples. Connection creation is privilege-gated: DATAMANAGEMENT plus
CAN_CREATE_OR_EDIT_CONNECTIONS
under RBAC.

---

3. Integrations catalogue [VERIFIED]

Data platforms (30+ live-query connections): Snowflake, Databricks, BigQuery, Redshift, Azure Synapse, Athena, Aurora, RDS, ClickHouse, Denodo, Dremio, Generic JDBC, AlloyDB, Cloud SQL, Looker,
Mode, MySQL, Oracle, PostgreSQL, Presto, SAP HANA, SingleStore, SQL Server, Starburst, Teradata,
Trino. (developers.thoughtspot.com/docs/connections)

ThoughtSpot Sync — destinations: Slack, Google Sheets, Microsoft Teams, Excel, Salesforce, ServiceNow, HubSpot, Gainsight, Zoho. The UI shape is worth copying: Data workspace → Sync, three tabs — Destinations (manage app connections), Pipelines (create/edit/schedule), Activity (execution history and logs). Users push from Answers or custom SQL views. Limits are surfaced in docs: >50,000 rows may time out; unsupported on Answers v1 / Liveboards v1. (docs.thoughtspot.com/cloud/latest/thoughtspot-sync)

Analyst Studio: dbt data-freshness metadata inline in Reports; Google Sheets; semantic-layer integrations extending metrics from dbt and Looker; webhooks.

Slack has a dedicated page whose governance line is precisely Spiff's pitch: ask NL questions in channels/threads via Spotter, subscribe answers/dashboards to channels, route KPI alerts — and
"Slack responses honor ThoughtSpot permissions and row-level security, so users only see what they're allowed to see." Also a ThoughtSpot Connected Sheets Workspace Marketplace add-on, DataSpot (Databricks packaging), and a Snowflake Cortex Agents ↔ Spotter integration over MCP.

Presentation pattern: integrations are not one flat catalogue but split by job — connections (admin/data workspace), sync destinations (three-tab console), custom actions (developer portal), marketplace add-ons (external stores).

---

4. ThoughtSpot's MCP server [VERIFIED]

Sources: developers.thoughtspot.com/docs/mcp-integration · thoughtspot.com/blog/introducing-agentic-mcp-server · github.com/thoughtspot/mcp-server

Yes — the Spotter MCP Server, hosted (no local install) at agent.thoughtspot.app.

Endpoints: OAuth apps /mcp (Streamable HTTP) and /sse (legacy); bearer-token apps /token/mcp and /token/sse. Version pinning via ?api-version=latest | beta | YYYY-MM-DD.

Tools: ping (test connectivity/auth) · getRelevantQuestions · getAnswer · createLiveboard (from a list of answers) · getDataSourceSuggestions. Spotter 3 (May 2026) adds session tools: create_analysis_session, send_session_message, get_session_updates.

Resources: a datasources resource listing "ThoughtSpot Data models the user has access to" — the resource list is itself permission-scoped.

Auth: OAuth 2.1/2.0 with Dynamic Client Registration; manual registration at agent.thoughtspot.app/clients for clients without DCR; bearer TS_AUTH_TOKEN for API-style integrations. Clients: Claude, ChatGPT (OpenAI integration GA, Deep Research early preview), Gemini, any MCP-capable agent. Claude Desktop uses npx mcp-remote <url>.

Governance claim: "all the permissions and access controls you've already established in ThoughtSpot are automatically respected when your agents access data."

---

5. MCP itself [VERIFIED]

Sources: modelcontextprotocol.io/docs/getting-started/intro · /docs/2026-07-28/learn/architecture · /learn/server-concepts · /specification/2026-07-28/basic/authorization · .../security_best_practices

Participants. An MCP Host (the AI app) instantiates one MCP Client per MCP Server. Local servers use stdio; remote servers use Streamable HTTP. "Connector" is Anthropic's product word for a server a user has added to Claude.

Primitives — and who controls each, the governance-relevant part:

Primitive	Controlled by	Methods
Tools — functions the model calls to act	Model	tools/list, tools/call
Resources — read-only context, URI-addressed	Application	resources/list, resources/templates/list, resources/read
Prompts — parameterised templates	User	prompts/list, prompts/get

Tools carry name, title, description, inputSchema (JSON Schema). Resources are URI-addressed (calendar://events/2024) with templates (travel://activities/{city}/{category}) and parameter completion. Prompts surface as slash commands. As of protocol 2026-07-28: server/discover for capability discovery, opt-in notifications via subscriptions/listen, elicitation (server asks the user for input/confirmation); sampling and logging deprecated.

MCP's own stated consent UX expectations (server-concepts): "Tools may require user consent prior to execution." Applications should implement displaying available tools in the UI so users can decide availability per interaction; approval dialogs for individual tool executions; permission settings for pre-approving safe operations; activity logs that show all tool executions with their results.

Authorization. OAuth 2.1 + PKCE. The MCP server is an OAuth resource server and MUST implement Protected Resource Metadata (RFC 9728) for authorization-server discovery. Client identity now prefers Client ID Metadata Documents; DCR is deprecated. resource (RFC 8707) MUST be sent so tokens are audience-bound. Errors: 401 unauthorized, 403 insufficient scope, 400 malformed. A 403 carries WWW-Authenticate: Bearer error="insufficient_scope", scope="...", driving a step-up authorization flow where the client re-authorises with the union of old and newly challenged scopes.

Scope minimisation is a named risk: publishing all scopes in scopes_supported, wildcard scopes (, all, full-access) and bundling unrelated privileges cause "consent abandonment: users decline dialogs listing excessive scopes."

Consent UI requirements (security_best_practices) are concrete and a good spec to design to. The consent page MUST: identify the requesting client by name; display the specific scopes requested; show the registered redirect_uri; add CSRF protection; prevent iframing. Token passthrough is forbidden — a server MUST NOT accept tokens not issued for it.

---

6. How Claude presents connectors [VERIFIED]

Sources: claude.com/docs/connectors/directory · /verification · /custom/remote-mcp · support.claude.com/en/articles/11176164

- One catalogue serves Claude.ai, Desktop, mobile, Claude Code and Cowork. Browse at Customize > Connectors; admins at Organization settings > Connectors.
- Three trust labels: Verified (checkmark; tested for quality/compatibility — explicitly "not a security audit"), Community (screened, not reviewed in depth; label in the directory plus a pre-connect reminder), Custom (self-added, unreviewed). The label affects discovery and display, not capability: "once connected, a community connector has the same capabilities and access as any connector you grant."
- Suggested Connectors: in-chat recommendations when relevant. Ranking is usage-based.
- Team-plan request flow: members without permission see Request → Requested; admins get a "Requested by your team" section plus a Notifications > Requests tab with a count badge, and can enable or dismiss. The requester is shown the outcome.
- Add custom connector dialog, field by field: Name; Remote MCP server URL (Claude probes it and pre-fills detected auth, marked "Detected"); Authentication — Always required / Required when the server asks / None; OAuth client — Anthropic's hosted client metadata (recommended) / register automatically (DCR) / your own client ID; Request headers (beta, up to 4, each with a Required flag, write-only after save); Advanced > Transport.
- Per-conversation enablement: "+" button or "/" → Connectors → toggle services on.
- Tool access mode: Auto (default) or On demand.
- Per-tool permission, three states: Always allow / Needs approval / Blocked.
- Lifecycle honesty: auth settings cannot be edited after adding — remove and re-add, and members must reconnect. If a provider changes its endpoint, existing connections keep working but reclassify as "Custom".
- Publisher side: submission portal, review criteria, post-publication health and usage dashboard.

---

7. MCP connector UX: the anatomy [DESIGN REASONING]

An exhaustive element list for a connector experience worth mocking. Elements marked ★ are the ones most teams skip and most users need.

A. Directory / catalogue view
1. Search field + category filters (Data, Comms, Dev, CRM…), and a "Suggested for you" rail.
2. Connector card: icon, name, publisher org (not just the product), one-line description, trust badge (Verified ✓ / Community / Custom), install state (Connect / Connected / Requested), and ★ a capability hint chip — "Reads data" / "Writes data" / "Acts on your behalf". Users decide from the card, so make risk legible there.
3. ★ Usage signal and last-updated date — staleness is a security signal.
4. Members without rights see Request, not a dead Connect button.

B. Detail page (pre-auth)
5. Publisher identity block: legal org name, domain, support link, privacy policy.
6. What it does, in the user's language; 2–3 example prompts.
7. ★ Full tool list before connecting — name, plain-English description, read/write/destructive tag. The single most important element: the tool list is the permission surface.
8. ★ Data-flow statement: where data goes, who operates the server.
9. Endpoint URL, transport, protocol/API version, changelog link.
10. Inline trust-label explainer ("Verified means tested for quality, not a security audit").

C. Auth step
11. Auth-mode indicator (OAuth per user / on demand / none / shared API key).
12. ★ Identity confirmation: which account you're connecting as, with a switcher. Governance dies when people silently connect a service account.
13. Scope/consent list: one row per scope — human-readable label, technical scope string beneath, read/write icon, "why this is needed" tooltip.

14. ★ Show the redirect URI / destination host — required by the MCP spec, and the anti-phishing element.
15. Explicit Allow / Cancel, never pre-ticked.
16. ★ Step-up consent modal for later insufficient_scope challenges: "Spiff needs one more permission — Write to shared folders — to finish this. Grant / Not now."

D. Post-connect management
17. Connector row: status dot (Connected / Needs reauth / Error / Disabled), connected-as identity, connected-on date.
18. Per-tool permission control: Always allow / Needs approval / Blocked. Group read vs write; default every write to Needs approval.
19. Per-conversation toggle + Tool access: Auto / On demand.
20. Resources browser (which data models it can see); prompts as slash commands.
21. Reconnect, Edit, Remove (warning that members must reconnect).

E. In-flight consent
22. Tool-call approval card in the transcript: tool name, plain-English intent, the actual arguments rendered readably, target system, Allow once / Always allow / Deny.
23. ★ Collapsible post-execution receipt: what ran, what returned, how long, which identity.

F. Org policy (admin)
24. Org connector list: Enabled/Disabled, allow-listing, default per-tool policy pushed to members.
25. Requests inbox with count badge; approve/dismiss with a note back to the requester.
26. ★ Policy toggles: block Custom connectors; Verified-only; forbid shared-credential auth; re-consent every N days.

G. Observability & health
27. Usage log: timestamp, user, connector, tool, argument summary, outcome, duration, rows touched. Filterable, exportable.
28. Health panel: uptime, error rate, p95 latency, last successful call, version drift.
29. ★ Change alerts: a tool list that changed since consent raises a banner and optionally re-gates approval. The docs warn tools "can change after review" — design for it.

H. Error & degraded states
30. Distinct, differently-worded states for: not connected · token expired (Reconnect) · insufficient scope (Grant permission) · disabled by admin (Request access) · server unreachable (Retry + status link) · blocked by policy (name the policy) · rate-limited (when to retry) · partial result (what was omitted and why).
31. Never render an auth failure as an empty result. "No data" and "not allowed to see the data" must look different.

---

8. Spiff as an MCP server [DESIGN REASONING]

If Spiff exposes itself to Claude/Copilot/ChatGPT, the tool set should mirror Spiff's governance model, not a generic SQL surface. Never expose raw query execution.

Read tools (default: Always allow)
- list_reports — reports the calling user can see: name, owner, description, last run.
- describe_report — schema, filters, definitions, freshness, owner, certification status.
- run_report — execute a defined report as the calling user; returns rows + a scope banner.
- search_metrics — governed metric definitions with their canonical business meaning.
- explain_answer — lineage of a number: source, filters, transformations, run-as identity.

Write / action tools (default: Needs approval)
- create_automation — schedule a report. Approval card must show cadence, recipients, run-as.
- share_report — and the card must state plainly: sharing organises, it never widens access; each viewer's copy re-runs scoped to them.
- clone_automation — the sharing primitive; card states the clone will run as its new owner.

Resources: spiff://reports/{id}, spiff://metrics/{name}, spiff://datasets/{id} — each

list already filtered to the caller's permissions, mirroring ThoughtSpot's datasources.

Prompts: "Explain this number", "Build me a weekly GST summary", "Who can see this report?"

The governance/consent screen for Spiff should show:
1. Running as — the resolved identity, with org/tenant, and an explicit statement that every run re-checks identity and permissions at execution time.
2. Data scope — the datasets and row-level filters this identity resolves to, listed concretely ("GST division, UK entity, 2024–2026"), not as an abstract role name.
3. Tool list split read vs write, with write tools pre-set to Needs approval.
4. What the external AI client will receive — aggregated results only, no raw extracts; and whether results leave the tenant.
5. What it cannot do — an explicit denial list ("cannot widen sharing, cannot alter permissions, cannot query outside the semantic model"). Stating the ceiling builds more trust than listing the grants.
6. Audit statement — every call is logged against this identity, with a link to the log.
7. Expiry & revocation — consent duration, and a one-click Revoke that is not buried.

---

9. Design lessons for Spiff [DESIGN REASONING]

1. The tool list is the permission UI. Show every tool, in plain English, before the user connects — not after. Nobody reads OAuth scope strings; they do read a 6-row tool table.
2. Three permission states beat a binary. Copy Claude's Always allow / Needs approval / Blocked per tool, and default every write to Needs approval. It makes governance feel adjustable rather than absolute.
3. Split read from write visually, everywhere. In the tool list, the consent screen, the approval card and the log. This one distinction carries most of the perceived safety.
4. Show "running as" on every surface. Spiff's core invariant only becomes real when the identity is visible on the answer, the schedule, the shared copy, and the consent screen.
5. Make the ceiling explicit. A "what this cannot do" block is a differentiator; almost no product ships one, and it is exactly Spiff's story.
6. Design the step-up moment. Least-privilege means a mid-task permission prompt is normal. Give it a first-class modal that names the one missing permission and the task it unblocks.
7. Consent lists must be short. MCP's own docs name "consent abandonment" from oversized scope lists. Ask for a minimal starting scope and elevate on demand.
8. Copy ThoughtSpot Sync's three-tab console — Destinations / Pipelines / Activity — for any Spiff automations area. Configuration, schedule and history are three different questions.
9. Activity log is a product feature, not an admin afterthought. MCP explicitly lists activity logs as a trust mechanism. Give end users their own run history, not just admins.
10. Differentiate "no data" from "not permitted." Identical empty states are the fastest way to destroy trust in a permission-scoped tool.
11. Design for tool drift. Connectors change after approval. A "this connector added 3 tools — review" banner is cheap and high-credibility.
12. Trust badges must be honest. Anthropic ships "Verified means tested, not a security audit." Spiff should be equally blunt about what a "certified" report does and does not guarantee.
13. Requests are a flow, not a dead end. Request → Requested → admin inbox with badge → outcome shown back turns governance from refusal into routing.
14. Approval cards must render real arguments. "Spiff wants to run a tool" is useless; "Run GST Weekly Summary, filtered to UK entity, as Oren Alazraki" is a decision a human can make.